

Wenn die AI lügt, liegt es am Kontext: vier Fehlermodi in sieben Schritten zähmen

Worauf das aufbaut: Diese Unterlage ist eine eigene Aufbereitung und Einordnung des Videos von **Nate Herk**. Die Gedanken stammen von dort, Auswahl, Gliederung, Beispiele und Kommentierung sind meine. Wörtliche Zitate sind als solche gekennzeichnet, Übersetzungen aus dem Englischen sind eigene.
Originalquelle: <https://www.youtube.com/watch?v=Ek1NBfnnTH0>

Kern-These

Wenn dein AI-Betriebssystem nicht sauber organisiert ist, fängt dein Agent an zu halluzinieren, nicht aus böser Absicht, sondern weil er nur zeigt, was in seinem Kontext liegt. Herk nennt vier Fehlermodi, die dafür verantwortlich sind: Poisoning (ein falscher Fakt liegt zwischen den richtigen), Bloat (zu viel Material, die Nadel geht im Heuhaufen unter), Confusion (eine Lücke wird erfunden statt benannt) und Clash (zwei Quellen widersprechen sich, und die AI weiß nicht, welche gilt). Dazu kommt eine zweite Unterscheidung, die fast noch wichtiger ist: Expertise-Kontext (das, was immer geladen sein muss) gegen Situations-Kontext (das, was nur im Moment gebraucht wird). Wer diese sechs Konzepte kennt, kann sein System so bauen, dass die vier Fehlermodi seltener auftreten, und so prüfen, dass sie auffallen, bevor sie Schaden anrichten.

Flip: Der volle Ordner sammelt alles in einem Mega-Dokument, Regeln, Fakten und Wissen ungetrennt, und merkt den Bloat erst, wenn die Antworten schlechter werden. Der saubere Ordner hält eine schlanke Router-Datei, segmentiert sein Wissen nach Themen und lässt sich regelmäßig selbst prüfen, read-only, nie automatisch reparierend.

Landkarte: 7 Schritte, 3 Bewegungen

Bewegung A · Verstehen (Schritte 1 bis 2) → Bewegung B · Strukturieren (Schritte 3 bis 5) → Bewegung C · Am Laufen halten (Schritte 6 und 7)

- Erkenne die vier Fehlermodi
- Trenne Expertise von Situations-Kontext
- Mach deine Router-Datei zum reinen Wegweiser
- Segmentiere dein Wissen, bevor es zum Grab wird
- Lass die AI sich selbst prüfen, nie selbst reparieren
- Automatisiere wiederkehrende Datenpflege

- Erzwingen das Backtrack-Protokoll bei jedem Fehler

Schritt 1 · Erkenne die vier Fehlermodi (A)

Herk benennt vier Gründe, warum ein Agent etwas Falsches liefert. Poisoning heißt, ein falscher Fakt sitzt zwischen den richtigen, und die AI liefert ihn selbstbewusst aus, weil er einfach im Kontext lag. Bloat heißt, es liegt so viel Material vor, dass die relevante Information im Heuhaufen untergeht, Stichwort Context Rot und Nadel im Heuhaufen. Confusion heißt, ein Fakt fehlt oder ist irrelevant, und die AI füllt die Lücke lieber mit einer erfundenen Antwort, statt „weiß ich nicht“ zu sagen. Clash heißt, zwei Quellen widersprechen sich (im März galt „immer Rückerstattung“, im Juni „nie Rückerstattung“), und die AI weiß nicht, welcher Quelle sie vertrauen soll.

- **O-Ton:** „Poisoning means you have a false fact somewhere in the context.“
- **Werkzeug · Die Vier-Fehlermodi-Checkliste:** (1) Poisoning, Fix ist Verifikation, eine Web-Suche oder zwei, ein Abgleich mit der Live-Datenbank, bei Unsicherheit ein Mensch in der Schleife. (2) Bloat, Fix ist Segmentieren und die Trennung aus Schritt 2. (3) Confusion, Fix ist eine fehlende oder irrelevante Stelle explizit als Lücke markieren lassen, nicht füllen lassen. (4) Clash, Fix ist Datierung und eine klare Vorrang-Regel, welche Quelle bei Widerspruch gewinnt.
- **Falle & Fix:** Falle, nur reagieren, wenn ein Fehler bereits sichtbar geworden ist. Fix, die vier Modi als stehende Checkliste behandeln und routinemäßig gegenprüfen, nicht erst beim Symptom nachdenken.
- **Reflexion:** Welcher der vier Fehlermodi ist in deinem eigenen System zuletzt aufgetaucht, und wie hast du ihn bemerkt? Prüfst du aktuell nur, wenn ein Fehler schon passiert ist, oder auch routinemäßig, bevor einer passiert?

Schritt 2 · Trenne Expertise von Situations-Kontext (A)

Die zweite Unterscheidung erklärt Herk mit dem Bild von Schulleitung und Lehrkraft, die gemeinsam einen Sitzplan bauen. Die Schulleitung kennt die generellen Regeln, wo die Tafel steht, wie ein Klassenzimmer läuft, das ist Expertise-Kontext, immer geladen, wie ein System-Prompt. Die Lehrkraft kennt jedes einzelne Kind, wer schlecht sieht und vorne sitzen muss, welche zwei sich gegenseitig zum Lachen bringen, das ist Situations-Kontext, nur bei Bedarf geladen, genau dann, wenn er gebraucht wird.

- **O-Ton:** „Expertise context is the rulebook, your policies, your pre-loaded information that needs to be in every single run.“
- **Werkzeug · Der Rollen-Test:** (1) Frag bei jedem Fakt, brauche ich das immer, oder nur jetzt gerade? (2) Expertise-Kontext gehört ins Routing, dauerhaft geladen. (3) Situations-Kontext

wird per Live-Lookup nachgezogen, genau dann, wenn die Frage kommt, nie dauerhaft vorgehalten.

- **Falle & Fix:** Falle, Situations-Kontext dauerhaft vorhalten, weil er „auch mal nützlich sein könnte“. Fix, nur Expertise-Kontext bleibt immer geladen, alles Situative wird just-in-time gezogen, sonst entstehen genau die Bloat- und Confusion-Modi aus Schritt 1.
- **Reflexion:** Welcher Fakt in deinem System wird gerade dauerhaft vorgehalten, obwohl er eigentlich nur situativ gebraucht wird? Wo fehlt dir umgekehrt Expertise-Kontext, den du eigentlich immer griffbereit haben solltest?

Schritt 3 · Mach deine Router-Datei zum reinen Wegweiser (B)

Herks eigene Haupt-Router-Datei liest sich fast wie ein Inhaltsverzeichnis: hier liegt die Wissensbasis, hier der Hot-Cache, hier der Index, hier der Fallback, hier die Tools, hier die Vorlagen, hier die Projekte. Kein langer Fließtext, keine tief verschachtelte Anleitung, sondern eine Tabelle aus Fragen und Wegweisern. „Wenn du das brauchst, geh dort hin.“

- **O-Ton:** „This main CloudMD that I use up here, I treat this almost purely as a router.“
- **Werkzeug · Die Router-Probe:** (1) Öffne deine Router-Datei und zähl grob, wie viele Sätze Wegweiser sind und wie viele eigentlich Inhalt, der woanders hingehört. (2) Bau eine Tabelle, Wissensbasis-Ort, Hot-Cache, Index, Fallback, Tools, Vorlagen, Projekte. (3) Halte die Datei kurz, sie ist Inhaltsverzeichnis, kein Nachschlagewerk.
- **Falle & Fix:** Falle, die Router-Datei wird zum Sammelbecken für jede neue Regel, bis sie ein Monster-Dokument ist. Fix, Kurzform im Router, Detail in ausgelagerte Dateien, die der Router nur bezeichnet.
- **Reflexion:** Wie viele Sätze deiner eigenen Router-Datei sind Wegweiser, wie viele sind eigentlich Inhalt, der woanders hingehört? Wann hast du deine Router-Datei zuletzt wirklich schlank gehalten, statt nur eine weitere Zeile anzuhängen?

Schritt 4 · Segmentiere dein Wissen, bevor es zum Grab wird (B)

Herk hatte ursprünglich eine einzige Master-Wiki für alles. Bis sein eigenes System ihm vorschlug, zwei wiederkehrende Datentypen, YouTube-Transkripte und Meeting-Transkripte, in eigene Wikis zu trennen. Weniger Bloat, weniger Confusion, schnellere und günstigere Antworten, weil weniger Tokens durchsucht werden müssen. Bei Klienten-Wissen zieht er dieselbe Linie, intern (Kontext, Preise, Verlauf) getrennt von extern (die tatsächlichen Deliverables im Kunden-Repo), beides bleibt im Routing verzeichnet, aber getrennt gepflegt.

- **O-Ton:** „You might as well just split those up so that it makes it easier for me to search through data.“

- **Werkzeug · Die Segmentierungs-Frage:** (1) Wächst ein Wissens-Cluster stetig und ist thematisch klar abgrenzbar? Dann eigene Wiki-Datei oder eigener Ordner. (2) Klienten-Wissen, intern getrennt von extern, beides im Routing referenziert. (3) Prüf-Frage, würdest du diese Datei in deinem eigenen Ordner-Explorer ohne Suche wiederfinden? Wenn nein, umsortieren.
- **Falle & Fix:** Falle, alles in einem Master-Ordner belassen, weil Aufteilen erstmal Arbeit macht. Fix, sobald ein Knoten wächst und thematisch klar abgrenzbar ist, sofort abspalten, nicht warten, bis er zum Grab geworden ist.
- **Reflexion:** Welcher Ordner in deinem System ist gerade dabei, zum Wissens-Grab zu werden? Fändest du die letzte wichtige Datei, die du angelegt hast, ohne Suche wieder?

Schritt 5 · Lass die AI sich selbst prüfen, nie selbst reparieren (B)

Herks Audit-Skill liest das gesamte Projekt, prüft Routing-Integrität (zeigt jeder Verweis auf etwas, das wirklich existiert), Index-Wahrheit (stimmt der Index mit der Platte überein), Frische (ist ein Datenfeed aktuell, driftend, eingefroren oder bewusst on demand), Bloat und Duplikate, Hygiene und Kontext-Platzierung. Das Ergebnis ist ein Fund-Report mit Fix-Vorschlägen, „hier sind zehn Dinge, hier sind zehn Korrekturen, soll ich sie machen, ja oder nein“. Der Skill selbst rührt nichts an.

- **O-Ton:** „Read only, never fix, rename, or delete. Just give a report of what needs to be changed.“
- **Werkzeug · Der Selbst-Audit-Rhythmus:** (1) Read-only Checks, Routing-Integrität, Index-Wahrheit, Frische, Bloat und Duplikate, Hygiene, Kontext-Platzierung. (2) Das Deliverable ist ein Fund-Report mit vorgeschlagenen Fixes, nie eine automatische Heilung. (3) Wöchentlich oder monatlich laufen lassen, nicht nur bei Verdacht.
- **Falle & Fix:** Falle, dem Audit erlauben, gefundene Probleme gleich selbst zu reparieren. Fix, read-only Report, Freigabe bleibt bei dir, genau wie es das Approval-Gate ohnehin verlangt.
- **Reflexion:** Wann hast du deinen Selbst-Audit zuletzt laufen lassen, und was hat er gefunden? Was würde passieren, wenn dein Audit-Skill morgen automatisch reparieren dürfte, statt nur zu melden?

Schritt 6 · Automatisiere wiederkehrende Datenpflege (C)

Manche Daten sind kein Situations-Kontext, sondern ein fester Takt. Jeden Montag ein Q&A, jeden Dienstag ein Leadership-Meeting. Statt darauf zu hoffen, dass man sich ans Nachtragen erinnert, richtet Herk Crons ein, die diese Feeds automatisch ziehen. Vergisst er einmal, nachzutragen, holt der Cron es trotzdem.

- **O-Ton:** „Why not just set up crons to automatically pull that stuff in there, so that if I for some reason forget, I don't have to worry about it.“
- **Werkzeug · Der Cron-Check:** (1) Frag dich, ist das ein Datenfluss, der auf einem festen Takt passiert? (2) Wenn ja, Cron oder Scheduler statt manuellem Erinnern. (3) Kein Ersatz für echten Situations-Kontext, nur für wiederkehrende, vorhersehbare Feeds.
- **Falle & Fix:** Falle, sich aufs eigene Erinnern verlassen, „ich trag das schon nach dem Meeting nach“. Fix, ein wiederkehrender Takt wird ein Cron, nicht ein Vorsatz.
- **Reflexion:** Welche wiederkehrende Datenquelle fütterst du gerade noch von Hand, obwohl sie einen festen Takt hat? Was vergisst du am ehesten nachzutragen, wenn eine Woche stressig wird?

Schritt 7 · Erzwingen des Backtrack-Protokoll bei jedem Fehler (C)

Der letzte Hack ist der unscheinbarste und der wirksamste. Merkst du, dass die AI etwas nicht gefunden hat, obwohl sie es hätte finden müssen, reicht es nicht, sie zu korrigieren. Lass sie zurückverfolgen, wo sie gesucht hat und warum sie dort nicht fündig wurde. Erst wenn sie ihren eigenen Fehler benennt und den besseren Pfad selbst formuliert, wird die Struktur nachgezogen, das Routing angepasst, Dateien verschoben. Das schlägt das bloße „mach das nie wieder“ um Längen.

- **O-Ton:** „Go look through what you did, where you searched, and help me figure out why you didn't find that data right away.“
- **Werkzeug · Das Backtrack-Protokoll:** (1) Fehler bemerkt, nicht nur korrigieren, sondern die Route zurückverfolgen lassen, wo wurde gesucht, was wurde übersehen. (2) Die AI benennt ihren eigenen Fehler und den besseren Pfad selbst. (3) Erst danach Routing oder Struktur tatsächlich nachziehen, nicht nur eine Ermahnung aussprechen.
- **Falle & Fix:** Falle, nur sagen „mach das nie wieder“, ohne die Route nachzeichnen zu lassen. Fix, die AI muss selbst benennen, wo sie gesucht hat und wo sie hätte suchen sollen, bevor irgendetwas repariert wird.
- **Reflexion:** Wann hast du zuletzt einen Fehler nur korrigiert, statt die AI ihre eigene Route nachzeichnen zu lassen? Was würde ein sauberes Backtrack-Protokoll bei deinem letzten wiederkehrenden Fehler zutage fördern?

Closing

Ordnung ist kein Nice-to-have, sie ist der Unterschied zwischen einer AI, die dir hilft, und einer, die dir vertrauensvoll etwas Falsches erzählt. Erkenne die vier Fehlermodi, trenne Expertise von Situation, halte den Router schlank, segmentiere dein Wissen, lass dich selbst prüfen statt

selbst reparieren, automatisiere die wiederkehrende Pflege, und verfolge jeden Fehler zurück, bevor du ihn korrigierst.

Der nächste kleine Schritt: Nimm dir eine einzige Datei vor, die gerade am ehesten nach Bloat riecht, und stell ihr die Rollen-Frage aus Schritt 2, gehört das hier rein, weil es immer gebraucht wird, oder nur, weil es gerade praktisch war?

Herkunft & Ehrlichkeit

AlVantum · Jay Manuzon · [Impressum](#) · [Datenschutz](#) · [coachjay.de](#)

Diese Unterlage wurde mit KI-Unterstützung erstellt und redaktionell geprüft. Weitergabe zum persönlichen Gebrauch erlaubt, gewerbliche Weiterverbreitung nicht.